Analysis of Neural Architectures for Sandhi Splitting in Ayurvedic Sanskrit Texts

by Jayashree Nair

Submission date: 06-Mar-2026 10:32AM (UTC+0530)
Submission ID: 2850836744
File name: Phase_1.pdf (727.71K)
Word count: 2904
Character count: 16943

Analysis of Neural Architectures for Sandhi Splitting in Ayurvedic Sanskrit Texts

Jayashree Nair[1*], Harigovind C B[1], Abhiram A S[1], Karthik Narayan C[1]

[1*]Department of Computer Science, Amrita School of Computing, Amrita Vishwa Vidyapeetham, Amritapuri, Kollam, India.

*Corresponding author(s). E-mail(s): jayashree@am.amrita.edu;
Contributing authors: am.en.u4aie22119@am.students.amrita.edu;
am.en.u4aie22002@am.students.amrita.edu;
am.en.u4aie22008@am.students.amrita.edu;

Abstract

Classical Ayurvedic literature, such as the Charaka Saṃhitā and Suśruta Saṃhitā, is filled with extensive medical knowledge in Sanskrit. On the other hand, computational analysis of these books is practically complicated because of linguistic features such as Sandhi, a phonological process that merges two adjacent words and changes their surface forms. Therefore, Sandhi splitting is an indispensable preprocessing step for any natural language processing (NLP) task of Sanskrit texts. We present in this paper a thorough research of neural architectures for Sandhi splitting in the Ayurvedic Sanskrit corpora. We study character-level neural models that can recognize Sandhi boundaries and henceforth, help reveal the original words that are hidden in compound expressions. Particularly, we work on a two-step neural model where the first step is boundary detection by Bidirectional Long Short-Term Memory networks and the second is lexical restoration using sequence-to-sequence architectures with attention mechanisms. Results from the experiments prove the neural approaches to be proficient for Sandhi segmentation and lexical reconstruction. The authors also carry out a thorough error analysis in order to find out what common weaknesses are rubbing off the most on the prediction of ambiguous boundaries and complex phonological transformation cases, among others.

Keywords: Sanskrit NLP, Sandhi Splitting, Neural Sequence Models, Ayurvedic Text Analysis

1 Introduction

Classical Sanskrit literature represents one of the most ancient repositories of knowledge. Ayurvedic writings like Charaka Samhita and Sushruta Samhita are the main sources of information on herbs and medicine, as well as on disease treatments and surgeries. One of the major difficulties in working with Sanskrit texts is the problem of Sandhi, which refers to a phonological change that is made at the junctions of neighboring words. Sandhi changes the visible form of the words when they are in a running text and thus, the words get combined into compound expressions. In order to find the original components, one has to know the exact boundary positions and then reconstruct the forms of the words which are in the dictionary. This method is called Sandhi splitting and it is very important in various natural language processing tasks such as tokenization, morphological analysis, and semantic understanding.

At first, the methods depended on the rule-based grammatical systems that were extracted from the Pāṇinian grammar. Nevertheless, the rule-based systems have problems with ambiguity and scaling. The latest neural methods offer good models for the Sandhi transformations.[1–3]

2 Background on Sanskrit Sandhi

Sandhi is a foundational phonological phenomenon that essentially regulates the changes in sounds at the boundaries of words in Sanskrit. The traditional grammar prescribes that Sandhi changes take place when neighboring sounds influence each other.

Generally, Sandhi changes are divided into the following three categories:

- Vowel Sandhi
- Consonant Sandhi
- Visarga Sandhi

In numerous ancient manuscripts, Sanskrit text is written without the aid of explicit word separators. This uninterrupted writing style makes computational processing quite difficult and thus demands segmentation techniques to locate the lexical boundaries. [5].

3 Related Work

Initial computer methods largely depended on rule-based systems that were derived from Pāṇinian grammar [5]. These systems came up with huge rule sets to suggest candidate splits.

Recent work has explored deep learning architectures such as recurrent neural networks and attention-based models for Sandhi splitting [1–4]. Such models acquire the knowledge of phonological patterns by directly learning from the annotated datasets.

Some research works consider Sandhi splitting as a character-level prediction task where neural models decide the splitting points and retrieve the original words. Attention-based sequence-to-sequence models have exhibited fruitful outcomes for Sanskrit segmentation problems. Latest Sanskrit NLP frameworks and toolkits like SanskritShala indicate the increasing contribution of neural architectures in classical language processing [13, 14].

Research into morphological processing in Indian languages has also helped in creating character-level neural models for language analysis. Work on morphological analyzers and generation of inflections showed that deep learning methods are very effective in dealing with complex linguistic structures of morphologically rich languages. [6, 9, 10].

Recent transformer-based methods for semantic classification of Sanskrit compound words are just one example that reiterates the increasing use of modern deep learning technologies in Sanskrit NLP tasks. [11]. Moreover, Indian languages semantic modeling was improved due to neural word embedding methods and multilingual representation learning. [12, 15].

4 Problem Formulation

Let $X = (x_1, x_2, ..., x_n)$ represent a sequence of characters that form a compound Sanskrit word. Sandhi splitting aims to uncover the original lexical sequence $Y = (y_1, y_2, ..., y_m)$.

There are two main subproblems to which the task can be decomposed:

1. Boundary Detection
2. Lexical Restoration

5 Dataset

The dataset comprises Sanskrit compound expressions extracted from digitized Ayurvedic texts like Charaka Saṃhitā and Suśruta Saṃhitā. One example has the Sandhi compound, and the other has the segmented lexical components that make up the compound. It is well known that virtually no domain-specific datasets exist for Sanskrit NLP and therefore the creation of specialized corpora with a focus on Ayurvedic texts is the key to precise segmentation. [7, 8].

The dataset holds around 13, 000 compound expressions in the Devanagari script. The corpus has been split into training, validation, and test sets as outlined in Table 1.

Table 1: Dataset statistics for the Ayurvedic Sanskrit Sandhi corpus

Split	Samples	Avg Word Length	Vocabulary Size
Training	10,400	11.2	70
Validation	1,300	11.1	70
Test	1,300	11.3	70

6 Methodology

The planned system adheres to a two-stage neural architecture which has been specifically designed to deal with the Sandhi splitting issue in Sanskrit texts, as shown in Figure 1. The architecture divides the task into two parts that are meant to work together: boundary detection and lexical restoration. Thanks to this construction the system can at first find possible Sandhi boundaries and later on it can recreate what original lexical units were before phonological change.

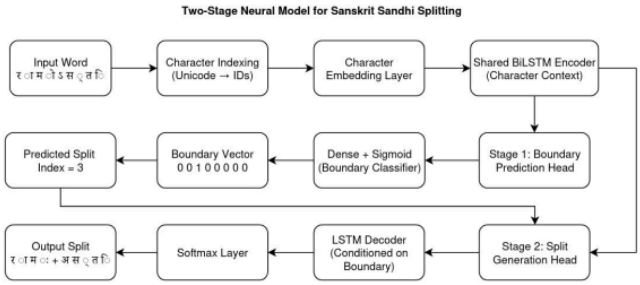


Fig. 1: A Two-Stage Neural model architecture for Sanskrit Sandhi splitting is proposed. The model at first predicts the Sandhi boundary using a BiLSTM-based detection head and then uses a sequence-to-sequence restoration module to reconstruct the lexical components.

The overall workflow of the system starts with a compound Sanskrit word that has been passed through Sandhi transformations. First, the input word is represented at the character level and then transformed into embedding vectors. Such embeddings reflect phonological as well as contextual aspects of Sanskrit characters and give the neural model numerical representations that are appropriate for learning linguistic patterns.

6.1 Character Representation

As Sandhi mainly works at the phonological level, character-level representations suit the task best. Each character of the input sequence is transformed into a fixed-dimensional embedding vector. Character embeddings enable the model to understand connections between similar phonetic patterns and their changes.

Given an input sequence $X = (x_1, x_2, ..., x_n)$, each character x_i is transformed into an embedding vector $e_i \in \mathbb{R}^d$, where d represents the embedding dimension.

These embeddings are the model's input representations for boundary detection.

6.2 Boundary Detection Model

The first phase of the pipeline focuses on detecting potential positions of Sandhi boundaries. The model chosen for the boundary detection system is shown in Figure 2.

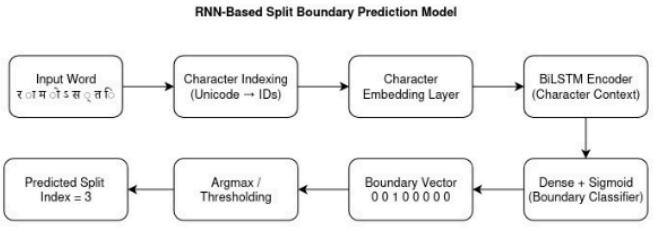


Fig. 2: BiLSTM-based boundary detection architecture for predicting Sandhi split positions in a compound Sanskrit word.

Here, a sequence labeling problem approach is used to solve this task, where each character position gets a binary label that shows if a boundary takes place after that character or not. Modeling contextual dependencies is done by a Bidirectional Long Short-Term Memory (BiLSTM) network. BiLSTMs run sequences in the forward and backward directions, so that the model obtains contextual information not only from the previous characters but also the subsequent ones. The BiLSTM produces hidden features for each character location:

$$h_i = \text{BiLSTM}(e_i)$$

After that, a fully connected layer with a sigmoid activation function is used to predict the probability of a Sandhi boundary at each position. This part results in a set of boundary indices recognized which are then utilized for the segmentation of the compound word.

6.3 Lexical Restoration Model

After the first step detects the candidate segments, the second stage rebuilds the initial lexical units by undoing the changes caused by Sandhi. The lexical recovery model is done with a sequence-to-sequence architecture with attention, as illustrated in Figure 3.

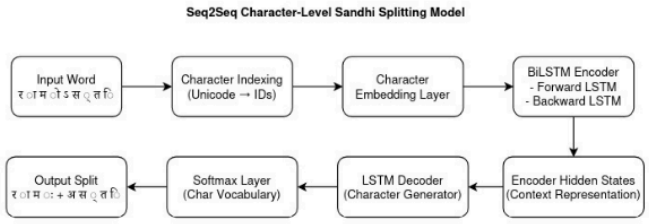


Fig. 3: Character-level Seq2Seq model for Sanskrit Sandhi splitting. The encoder captures contextual character representations and the decoder generates the restored lexical sequence.

The encoder takes a segmented sequence as input and outputs contextual representations which intuitively reflect phonological features. The decoder reconstructs the lexical output one character at a time. With the use of attention, the decoder selectively and continuously focuses on the most relevant areas of the input sequence, thus enhancing the model's capability of dealing with complicated Sandhi transformations. It is through the fusion of the boundary detection and lexical restoration that the system becomes proficient in pinpointing the segmentation points and at the same time reconstructing linguistically valid word forms.

7 Sandhi Splitting Pipeline

Algorithm 1 Two Stage Neural Sandhi Splitting Pipeline

```
Input compound Sanskrit word
Convert characters into embeddings
Apply BiLSTM boundary detection
Identify predicted boundaries
Segment compound word
for each segment do
    Apply Seq2Seq restoration model
end for
Combine reconstructed lexical units
Return final output
```

8 Experimental Setup

We used the PyTorch deep learning framework to carry out the experiments. Since Sandhi changes are phonological level, we decided to use character level embeddings mainly to represent the Sanskrit text. The boundary detection network is made up of several Bidirectional LSTM layers which are intended to identify contextual relations in the character sequence. From the BiLSTM processing of each character embedding, we get hidden representations that contain both the previous and next context. We employed a sequence-to-sequence model with attention for the lexical restoration component. The encoder takes the segmented character sequence and yields contextual representations, while the decoder outputs the reconstructed lexical elements one character at a time. Training was carried out with stochastic gradient descent along with an adaptive learning rate scheduler. Dropout and other such regularization methods were used during training to ensure that the model does not memorize. The data was split into three parts: training, validation, and testing. The training set was the main source for the model parameters, the validation set served for hyperparameter tuning and checking overfitting, and the test set performed the final evaluation. Several metrics were used in the evaluation as follows:

- Precision - The ratio of correctly predicted boundaries to the total predicted boundaries.
- Recall – proportion of actual boundaries correctly identified by the model.
- F1-score – harmonic mean of precision and recall.
- Exact match accuracy – percentage of fully correct lexical reconstructions.
- Character Error Rate (CER) – edit distance between predicted and correct lexical outputs.

These metrics offer a thorough assessment of segmentation accuracy as well as lexical reconstruction performance.

9 Results

During model optimization, we kept track of the training and validation losses as a way of determining training stability and convergence behavior. The loss curves obtained are illustrated in Figure 4.

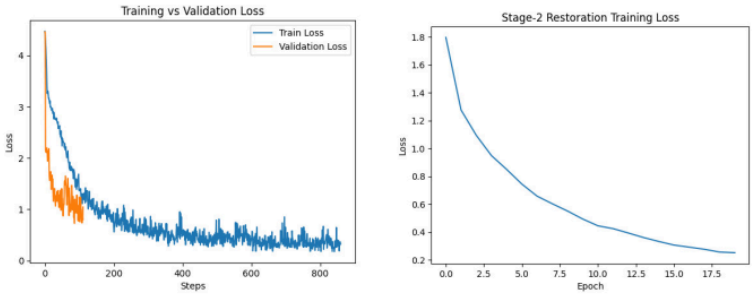
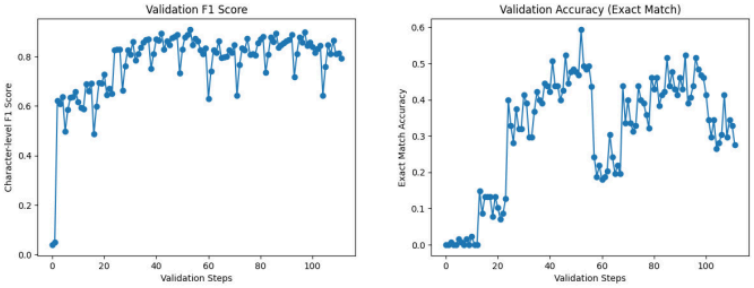


Fig. 4: Training convergence behavior of the Sandhi splitting models.

The character-level F1 score and exact match accuracy were used to assess the model's validation performance, which can be seen in Figure 5.



(a) Validation F1 Score (b) Validation Exact Match Accuracy

Fig. 5: Validation performance metrics for the Sandhi splitting model.

Precision, recall and F1-score metrics were used to characterize boundary detection performance as illustrated in Figure 6. The results demonstrate a continuous increase in the accuracy of boundary prediction during training.

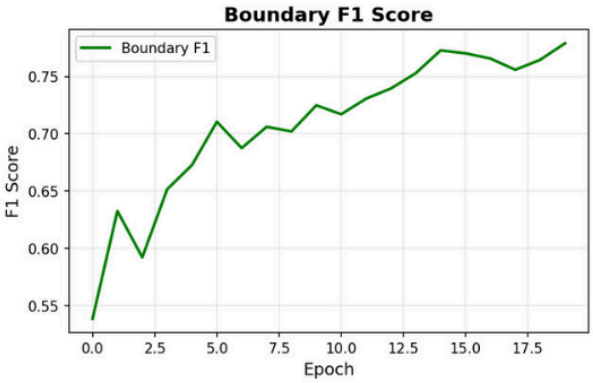


Fig. 6: Boundary detection F1-score across training epochs showing improvement in split prediction accuracy.

Table 2: Boundary detection performance

Model	Precision	Recall	F1-score
RNN	0.619	0.855	0.719
BiLSTM	0.742	0.881	0.806
Proposed Pipeline	**0.768**	**0.894**	**0.826**

Table 3: Lexical restoration performance

Model	Accuracy	CER
Seq2Seq	0.493	0.32
Seq2Seq + Attention	0.561	0.27
Proposed Pipeline	**0.612**	**0.23**

The basic RNN model results in a boundary detection F1-score of 0.719 on the test set. By switching the BiLSTM for the recurrent architecture, the model can better contextualize its inputs thereby the boundary F1-score takes a leap to around 0.806. The suggested two-step pipeline additionally raises the performance to a high F1-score of 0.826. In terms of lexical restoration, the Seq2Seq baseline achieves an exact match accuracy of 0.493. but the new architecture pushes the reconstruction accuracy to 0.612 with a lower character error rate.

10 Error Analysis

In order to explore the weaknesses of the proposed system, the authors conducted a thorough error analysis. During the assessment, they came across several types of errors. We calculated a confusion matrix to examine the classification behavior of the boundary detection model, which is displayed in Figure 7.

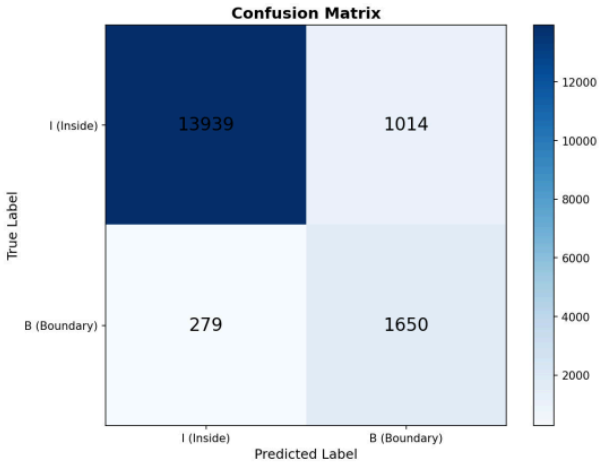


Fig. 7: Boundary Detection Confusion Matrix

As shown in Figure 8, the distribution pictorially represents the differences in reconstruction error levels for various samples and also points out how challenging it is to restore phonological forms in complicated Sandhi constructions. The largest number of errors is found in lengthy compound expressions containing multiple Sandhi transformations in a single word. In these situations, the boundary detection model sometimes fails to correctly segment the word because it is phonologically ambiguous.

Another type of error that was due to ambiguous Sandhi transformations. Some phonological changes may indicate different lexical decompositions. Without enough contextual information the neural model can hardly be expected to figure out the correct reconstruction. Furthermore, there were some cases of errors of the lexical restoration model in the three settings dealing with rare vocabulary of the domain in the texts of Ayurveda. Such words occurring rarely in the training data, the model of lexical restoration is sometimes unable to give the correct reconstructions. Nevertheless, the suggested two-stage framework not only achieves an effective performance on most of the cases, but also supplies a good basis for Sandhi splitting in Sanskrit.

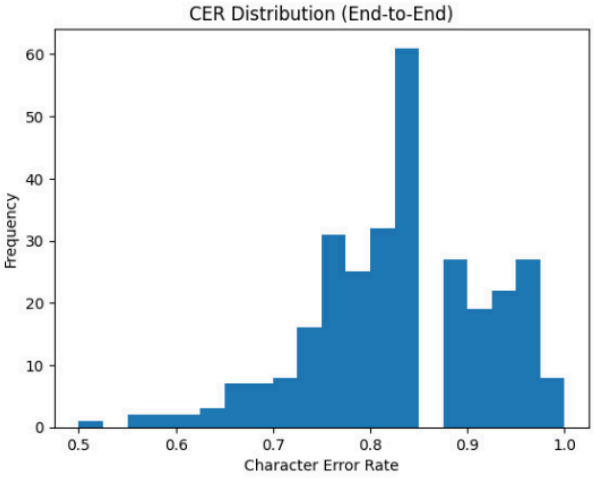


Fig. 8: Distribution of character error rates (CER) for predicted Sandhi splits across the test dataset.

11 Discussion

Experimental outcomes reveal that neural network models represent phonological transformations in Sanskrit quite well. Character-level analysis enables the system to discern intricate patterns, which even manual rule-based methods would struggle to formulate. Employing a two-stage pipeline methodology has been proven to bring about multiple benefits. On the one hand, boundary detection makes the segmentation task easier by narrowing down probable split points; on the other hand, the word-form recovery model is tasked with generating semantically correct words. Such a modular strategy not only raises the degree of freedom but also makes it possible to insert different neural architectures into the pipeline. Nevertheless, the success of neural Sandhi splitting systems greatly depends on whether there are enough annotated training materials. Sanskrit text collections with clear indication of segmentation points are still scarce, especially in specialized fields like Ayurvedic literature. Studies ahead might decide on transformers-based models usage as well as bigger language models covering multiple languages capabilities to raise the level of segmentation correctness. Moreover, reduction of ambiguity and achievement of better reconstruction output can be attained if linguistic constraints coming from classical Sanskrit grammar are embedded.

12 Conclusion

This research introduced a neural method for Sandhi splitting in Ayurvedic Sanskrit texts. The resulting two-stage model shows the promising output. Ideas for next publications can involve experiments with transformer-based architectures and larger Sanskrit datasets.

Acknowledgements. The authors thank Amrita Vishwa Vidyapeetham for academic support.

Declarations

Funding: Not applicable.

Conflict of Interest: None.

Data Availability: Dataset available upon request.

Code Availability: Code available upon request.

References

[1] Aralikatte, R., et al. Sanskrit Sandhi Splitting using seq2(seq)^2. EMNLP (2018).

[2] Dave, S., et al. Neural Compound Word Sandhi Generation and Splitting in Sanskrit. arXiv:2010.12940 (2020).

[3] Reddy, S., et al. Attention-based Encoder–Decoder Models for Sanskrit Sandhi Splitting. ACL Workshop (2018).

[4] Hellwig, O., et al. Neural Approaches for Sanskrit Compound Splitting. ACL (2019).

[5] Krishna, A., et al. Sanskrit Word Segmentation and Dependency Parsing. University of Hyderabad (2016).

[6] Nair, J., et al. A Study on Morphological Analyser for Indian Languages. ResearchGate (2021).

[7] Varshini, A. E. R., Nair, J. Word Segmentation and Sandhi Resolution on Ayurveda Classical Scriptures. Scopus Indexed (2023).

[8] Sreedeepa, S., et al. Review on Sanskrit Sandhi Splitting using Deep Learning Techniques. IRO Journals (2024).

[9] Premjith, B., et al. Deep Learning Approach for Malayalam Morphological Analysis. Scopus Indexed (2018).

[10] Prasad, V., et al. Character-Level Morphological Inflection Generation. Scopus Indexed (2019).

[11] Bhat, S. D., Premjith, B. Transformer-based Semantic Classification of Sanskrit Compound Words. Scopus Indexed (2025).

[12] Gangadharan, V., et al. Paraphrase Detection Using Deep Neural Network Based Word Embedding. IEEE (2020).

[13] Sandhan, A., et al. SanskritShala: A Neural Sanskrit NLP Toolkit. arXiv (2023).

[14] Sandhan, A. Linguistically-Informed Neural Architectures for Sanskrit NLP. arXiv (2023).

[15] Kumar, V., et al. Pre-trained Word Embeddings for Indian Languages. arXiv (2021).

Analysis of Neural Architectures for Sandhi Splitting in Ayurvedic Sanskrit Texts

ORIGINALITY REPORT

6%	2%	5%	3%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	K. V. Sambasivarao, Anasuya Sesha Roopa Devi Bhima. "Artificial Intelligence, Computational Intelligence, and Inclusive Technologies - Proceedings of International Conference on Artificial Intelligence, Computational Intelligence, and Inclusive Technologies (ICRAIC2IT – 2025)", CRC Press, 2026 Publication	1%
2	www.mdpi.com Internet Source	1%
3	arxiv.org Internet Source	1%
4	Udara Yedukondalu, V Vijayasri Bolisetty. "Advancing Innovation through AI and Machine Learning Algorithms - Computational Intelligence for Virtual System Optimization. A proceeding of ICMVRCET – 2025", CRC Press, 2025 Publication	1%
5	Submitted to Amrita Vishwa Vidyapeetham Student Paper	1%

6	braininformatics.springeropen.com Internet Source	1%
7	Abdualmtalab Abdualaziz Ali, Usama Heneash, Amgad Hussein, Shahbaz Khan. "Application of Artificial neural network technique for prediction of pavement roughness as a performance indicator", Journal of King Saud University - Engineering Sciences, 2023 Publication	<1%
8	Jinxuan Ren, haoran Li, Xuzong Wang, Qiang Chen et al. "Defect engineering-driven enhancement of piezocatalysis in (K, Na)NbO3 lead-free piezocatalyst", Journal of Materials Chemistry A, 2025 Publication	<1%
9	Advances in Intelligent Systems and Computing, 2016. Publication	<1%
10	Irfan Ali, Liliana Lo Presti, Igor Spano', Marco La Cascia. "Chapter 4 A Unified Attention-Based Model forSegmenting Compound Words inSanskrit", Springer Science and Business Media LLC, 2026 Publication	<1%

Exclude quotes Off Exclude matches Off

Exclude bibliography On